Module 15: Apex Triggers

Course Objectives

At the end of this course, you will be able to:

- Explain what are Triggers?
- Learn Trigger Syntax, Types of Triggers, Context Variables
- Perform use cases on Before Triggers
- Perform use cases on After Triggers
- Perform use cases on Trigger.old, Trigger.oldMap
- Perform use cases on Trigger Bulkification - considering Data loader
- Understand Trigger Best Practices
- Understand Order of Execution

15.1 Get Started with Apex Triggers

Definition

What is Trigger?

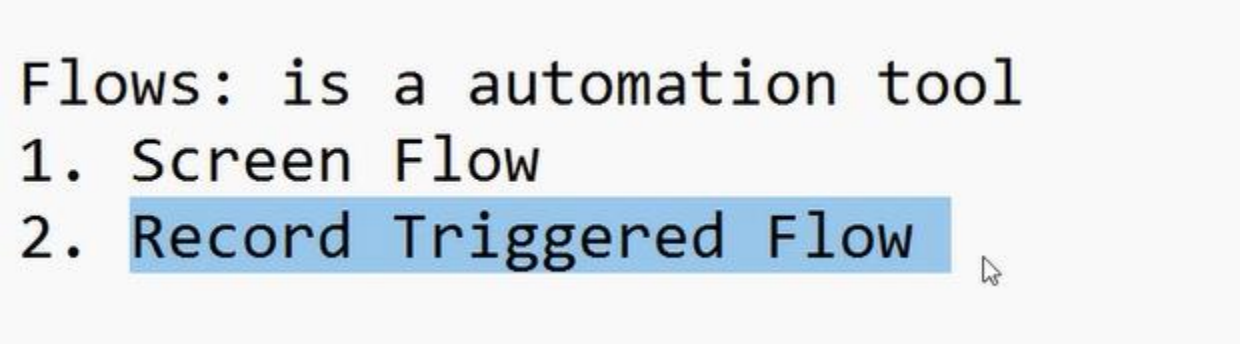
- A Trigger is a segment of code that executes whenever data in the underlying table is impacted by any Data Manipulation Language (DML) operations
- Apex triggers are programmatic components written in the Apex programming language
- Designed to execute specific actions before or after changes are made to Salesforce records

Why Trigger?

A trigger in Salesforce is used to automate processes and enforce business rules when data changes occur in the Salesforce database

Apex Triggers:

Trigger is a segment of code which is executed automatically based on the DML operations. (insert, update, delete, undelete)



Triggers can be used in place of validation Rules and Record triggered flow. If you can create a solution with the help of validation rules and record triggered flow, go for these tools. (Because here we don't have to write any code). If these tools are not enough to achieve the required solution, then go for apex triggers.

Some basic functionality of apex triggers:
1. Sending an email -->
2. Updating a field value

Syntax for Trigger Definition

Defining a trigger includes giving the trigger a name, specifies the object on which it operates, and defines the events that cause it to fire.

Syntax:

```
trigger TriggerName on ObjectName (trigger_events)
{
    //code-block
}
```

- Object Name - Represent the API name of SFDC objects
- Trigger Events - after insert, before insert, before update, after update, before delete, after delete and after undelete

15.2 Types of Triggers, Trigger Context Variables

Trigger Types

- Triggers can be divided into two types:

Before triggers can be used to update or validate record values before they are saved to the database

After triggers can be used to access field values that are set by the database (such as a record's Id or last updated field), and to affect changes in other records related to the Triggering record

```
Trigger Events: WE have 7 trigger events
1. Before Insert
2. Before update
3. Before Delete
4. After insert
5. After update
6. After delete
7. After undelete

Types of Triggers:
1. Before Triggers --> Trigger code will be executed before inserting,
updating, deleting any record from the database.
2. After Triggers --> Trigger code will be executed after inserting, updating,
deleting any record from the database.
```

Trigger Types & its operations

An Apex trigger can be executed "before" or "after" the following types of operations:

* Before undelete is not possible

Trigger Context Variables

- All triggers define implicit variables that allow developers to access runtime context about the event that invoked the trigger.
- The Trigger class provides the following context variables :

  - isInsert
  - isUpdate
  - isBefore
  - isAfter
  - isUndelete
  - isDelete
  - size
  - new
  - newMap
  - old
  - oldMap
  - isExecuting
  - operationType

```
1. Before Triggers --> Trigger code will be executed before inserting,
updating, deleting any record from the database.
Use Cases of Before Triggers:
        1. To validate the field values(just like validation rules)
        2. To update any field value on that object(Field update)
2. After Triggers --> Trigger code will be executed after inserting, updating,
deleting any record from the database.(WE have the record Id)
Use Cases of After Trigger:
        1. Sending an email
        2. Post to Chatter
        3. update fields of related objects
```

Trigger Context Variables: It is a variable used to hold some value, and it will be used only with trigger context.

1. IsInsert, IsUpdate, IsDelete, IsUndelete, isBefore, isAfter --> This holds either true or false. It is generally used in the If() condition, to check whether the record is inserted or updated or deleted.

2. new, newmap, old, oldmap
Trigger.new --> It holds the list of new version of records that are getting inserted or updated. (Means this will be used with insert/update events only)
Trigger.old --> It holds the list of old version of records that are getting updated or deleted.(Means this will be used with update/delete events only)
Trigger.newmap --> It holds the map of new version of records (Key-value pair)
Trigger.oldmap --> It holds the map of old version of records (Key-value pair)

Using Trigger Context Variables

```
trigger TriggerName on ObjectName__c (before insert, before update) {
    if (Trigger.isBefore) {
        if (Trigger.isInsert) {
            // code logic
        }
        if (Trigger.isUpdate) {
            // code logic
        }
    }
}
```

15.2 Types of Triggers, Trigger Context Variables

Create a Trigger on Candidate Object with Trigger.new and Trigger.old context variables. Create a candidate from UI with First Name as “Dhoni” and then update the existing record with First Name as “MS Dhoni”. Check the debug logs.

Solution:

CandidateTrigger.apxt

Code Coverage: None API Version: 60

```
1  trigger CandidateTrigger on Candidate__c (before insert,before update) {
2      System.debug('Trigger.new: '+Trigger.new);
3      System.debug('Trigger.old: '+Trigger.old);
4  }
```

On Insertion of Candidate record with First Name as “Dhoni”,
Line No. 2 - Record (Dhoni)
Line No. 3 - NULL

Update the existing inserted Candidate record with First Name as “M S Dhoni”,
Line No. 2 - Record (M S Dhoni)
Line No. 3 - Record (Dhoni)

To-Do

15.2 Types of Triggers, Trigger Context Variables

Create a Trigger on Position object with before insert, after insert, before update and after update events, also add debug statements on Trigger.new, Trigger.old, Trigger.oldMap and Trigger.newMap.

Create a Position record from UI with Name as ‘Senior Manager’, Job Description as ‘Senior Manager manages the Team’ and Status as ‘New’. Save the Record. Update the Status value to ‘Open’. Check the debug logs.

15.3 Before Triggers

Before Triggers Definition

- A "before trigger" is a type of trigger that allows to manipulate data before it is saved to the database.
- Before triggers fire immediately before a record is inserted, updated, or deleted.
- This is particularly useful for validating or modifying data before the database operation (insert, update, delete, etc.) is completed.
- The key advantage of using before triggers is that changes made to the records that are being processed without the need for a separate update statement.

Before Triggers Syntax

```
trigger TriggerName on ObjectName (before insert | before update | before delete) {        //
Trigger logic here
}
```

Example: (Basic Trigger Example without using Best Practices)

Populate the Position record with Status as 'New' on insertion

```
trigger PositionTrigger on Position__c (before insert) {
   for(Position__c pos:Trigger.new){
      pos.Status__c = 'New';
   }
}
```

Creating Handler Class

- **Logic-less Triggers:**
  Triggers themselves should primarily handle record access and context management. Move complex business logic to separate Apex classes for better organization and reusability.

- **Context-specific Handler Methods:**
  Create handler methods within trigger class that handle specific DML operations (e.g., beforeUpdateHandler, afterDeleteHandler). This improves code readability and organization. Delegate the logic from the trigger to a handler class. This makes the trigger more readable and maintainable. By default, Apex executes in the system context. Object permissions, field-level security, and sharing rules aren't applied for the current user. Can use the "with sharing" keyword in the class to specify that the sharing rules for the current user is applied in the business logic of the Trigger.

Example for Before Trigger with Handler Class

Populate the Position record with Status as 'New' on insertion with help of Trigger Handler class.

```
trigger PositionTrigger on Position__c (before insert) {
  PositionTriggerHandler.statusUpdate(Trigger.new);
}

public class PositionTriggerHandler {
   public static void statusUpdate(List<Position__c> newPosition){
      for(Position__c posRec: newPosition){
            posRec.Status__c = 'New';
      }
   }
}
```

Add Error() method

- The addError method in Apex Triggers is used to prevent DML operations from occurring.
- When addError is called on a record within a trigger, the operation is rolled back, and the specified error message is displayed to the user.
- This method is crucial for implementing custom validation logic directly within your triggers.

Using addError in Triggers:

- Identify the Record:
  Access the record which want to be prevented from being saved. This could be from the new or old list depending on your trigger context
- Validate and Add Error:
  Perform your validation logic within the trigger. If an error is found, use the addError method on the specific record object.

Example - addError() method

Prevent deletion of 'Open' Positions

```
trigger PositionTrigger on Position__c (before delete) {
   PositionTriggerHandler.preventDeletionOpenPosition(Trigger.old);
}

public class PositionTriggerHandler {
   public static void preventDeletionOpenPosition(List<Position__c> delPositionList){
      for (Position__c posRec : delPositionList) {
         if (posRec.Status__c == 'Open') {
               posRec.addError('Cannot delete the Open Position record');
         }
      }
   }
}
```

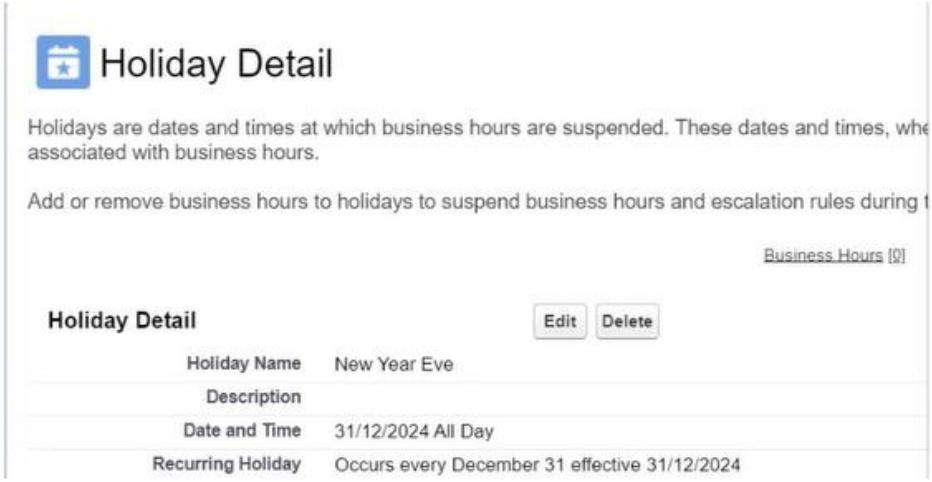
```
insert --> trigger.new
update --> trigger.new, trigger.old
delete --> trigger.old
```

Demo

15.3 Before Triggers

Position Start Date cannot be on a Company Holiday.

Create a Holiday record with the following details,



15.3 Before Triggers

Solution:

```
trigger PositionTrigger on Position__c (before insert, before update) {
    PositionTriggerHandler.preventInvalidPosition(Trigger.new,Trigger.oldMap);
}
```

```
public class PositionTriggerHandler {
    public static void preventInvalidPosition(List<Position__c> newPositionList, Map<ID, Position__c> oldPositionMap) {
        Set<Date> allHolidays = new Set<Date>();
        for (Holiday h : [SELECT ActivityDate FROM Holiday]) {
            allHolidays.add(h.ActivityDate);
        }
        for (Position__c posRec : newPositionList) {
            if (oldPositionMap?.get(posRec.Id).Start_Date__c != posRec.Start_Date__c) {
                if (allHolidays.contains(posRec.Start_Date__c)) {
                    posRec.Start_Date__c.addError('Position Start Date cannot be on Holiday');
                }
            }
        }
    }
}
```

P

15.4 After Triggers

After Triggers Definition

- “After triggers" are used to perform operations after a record has been saved to the database.
- The record itself is considered read-only within an after trigger.
- Can't directly modify the saved record's fields.
- These triggers are useful when need to access field values that are set by the system (such as Id for new records) or perform actions that depend on the record being committed to the database, like sending an email or updating related records.

After Triggers Syntax

```
trigger TriggerName on ObjectName (after insert | after update | after delete | after
                                   undelete)
{
    // Trigger logic here
}
```

Demo

15.4 After Triggers

Once the Position record is created with ‘Open’ Status, provide ‘Edit’ access to the Hiring Manager for the created record.

Note: Position object should have OWD as private & Hiring Manager is required for every ‘Open’ Position record

Solution:

```
trigger PositionTrigger on Position__c (after insert,after update) {
   PositionTriggerHandler.recordSharing(Trigger.new);
}
```


Demo

15.4 After Triggers


```
public class PositionTriggerHandler{
      public static void recordSharing(List<Position__c> newPositionList){
        List<Position__Share> lstPositionShareRecord = new List<Position__Share>();
        for(Position__c posRecord: newPositionList){
            if(posRecord.Status__c == 'Open' && posRecord.Hiring_Manager__c != NULL){
                Position__Share objShare = new Position__Share();
                objShare.AccessLevel = 'Edit';
                objShare.ParentId = posRecord.Id;
                objShare.UserOrGroupId = posRecord.Hiring_Manager__c;
                lstPositionShareRecord.add(objShare);
            }
        }
        if(!lstPositionShareRecord.isEmpty()){
            Database.insert(lstPositionShareRecord,false);
        }
    }
}
```

To-Do

15.4 After Triggers

Whenever a Position record is created with 'Open' Status, make Hiring Manager follower of the record.

15.5 Trigger.new vs Trigger.old, Trigger.oldMap

Trigger.new & Trigger.old Definition

- **Trigger.new**

  Returns a list of the new versions of the sObject records. Note that this sObject list is only available in *insert* and *update* triggers, and the records can only be modified in before triggers.

- **Trigger.old**

  Returns a list of the old versions of the sObject records. Note that this sObject list is only available in update and delete triggers.

Note:

- Delete supports only Trigger.old
- Insert supports only Trigger.new
- Update supports both Trigger.old & Trigger.new

Comparison

Variable	Trigger.new	Trigger.old	Trigger.newMap	Trigger.oldMap
Description	List of new sObject records being inserted or updated.	List of old sObject records before update or delete.	Map of record IDs to the new versions of sObjects.	Map of record IDs to the old versions of sObjects.
Availability	Before Insert, After Insert, Before Update, After Update	Before Update, After Update, Before Delete, After Delete	Before Update, After Insert, After Update	Before Update, After Update, Before Delete, After Delete
Use Case	Access and modify new records (only in Before triggers).	Access old record values for comparison or validation.	Access new records using their IDs for efficient lookups.	Access old records using their IDs for efficient lookups.

```
What is a Share Object?
For every custom object, we have a share object, for position__c we have
position__share. We can use this object in code to give different Access
permissions to different users.

Trigger.newmap --> it contains the map of new version of records.
It is available in insert and update triggers.

map<ID, sobject> -->
ID --> Record id
sObject--> the whole record.
```

15.5 Trigger.new vs Trigger.old, Trigger.oldMap

Solution:

trigger PositionTrigger on Position__c (before update) {
    PositionTriggerHandler.statusUpdate(Trigger.new,Trigger.oldMap);
}

```apex
public class PositionTriggerHandler {
    public static void statusUpdate(List<Position__c> updatePosition, Map<Id,Position__c> oldPosition){
        for(Position__c pos : updatePosition){
            if(oldPosition.get(pos.Id).Status__c == 'New' && pos.Status__c == 'Open'){
                FeedItem feedItem = new FeedItem();
                feedItem.ParentId = pos.Hiring_Manager__c;
                feedItem.Body = 'Position Status updated to Open';
                Database.insert(feedItem,false);
                System.debug('Feed Item is '+feedItem);
            }
        }
    }
}
```

15.6 Bulkification

Definition

- Bulkifying triggers in Salesforce is a best practice that ensures trigger can optimize Apex code written in triggers to handle large volumes of records efficiently.
- It helps to processes records in batches, and to avoid hitting governor limits by processing multiple records within a single transaction.
- It's a crucial concept for ensuring smooth performance, avoid hitting governor limits and avoid issues when dealing with a significant number of records.

Why Bulkify Triggers?

Without bulkification, triggers that process numerous records might exceed governor limits, leading to errors or unexpected behavior.

- **Governor Limits:** Salesforce imposes limits on the number of DML operations, SOQL queries, Heap size limits(6MB for synchronous transactions and 12MB for asynchronous transactions) and other resources per transaction. Bulkifying triggers helps you stay within these limits.
- **Efficiency:** Processing records in bulk is more efficient than processing each record individually, reducing the risk of performance issues.
- **Reliability:** Bulkified triggers are more robust and can handle different volumes of data without modifications.

Demo

15.6 Bulkification

Whenever an update happen on Account object, update the associated opportunity records with Stage as 'Needs Analysis'

Un-bulkified Version(Performance Issues)

```
trigger AccountTrigger on Account (after update) {
   for (Account accRec : Trigger.new) {
      for (Opportunity oppRec : [SELECT Id FROM Opportunity WHERE AccountId =
                                 :accRec.Id]) {
          oppRec.StageName = 'Needs Analysis';
        update oppRec;
   }
  }
}
```

Demo

15.6 Bulkification

Bulkified Version – Solution:

```
trigger AccountTrigger on Account (after update) {
    AccountTriggerHandler.updateStageOpportunity(Trigger.new);
}
```

```
public class AccountTriggerHandler {
    public static void updateStageOpportunity(List<Account> accountList){
        Set<Id> accountIds = new Set<Id>();
        for (Account accRec : accountList) {
            accountIds.add(accRec.Id);
        }
        List<Opportunity> opportunitiesToUpdateList = [SELECT Id, StageName FROM Opportunity WHERE AccountId IN :accountIds];
        for (Opportunity oppRec : opportunitiesToUpdateList) {
            oppRec.StageName = 'Needs Analysis';
        }
        Database.update(opportunitiesToUpdateList,false);
    }
}
```

Key Techniques for Bulkifying Triggers

- **Processing Records in Batches:** Instead of iterating through each record individually and performing actions, bulkified triggers operate on collections of records at once. This significantly reduces the number of database calls and DML statements needed.
- **Leveraging SOQL and DML on Collections:** Bulkified code utilizes SOQL queries to retrieve groups of records and performs DML operations (update, insert, delete) on those collections instead of single records. This optimizes database interactions.
- **Utilizing Maps and Sets:** Maps are efficient for storing and retrieving records based on their unique identifiers. Sets are useful for identifying distinct values within a collection. These data structures can enhance processing efficiency within triggers.
- **Avoiding Nested Queries:** Executing multiple SOQL queries in the body of loop for each record can be very inefficient. Bulkified triggers aim to minimize nested queries by strategically combining data retrieval and manipulation logic.

To-Do

15.6 Bulkification

Whenever a Position record is created, create an Interviewer record with Role as ‘General’ & associate it with Position accordingly.

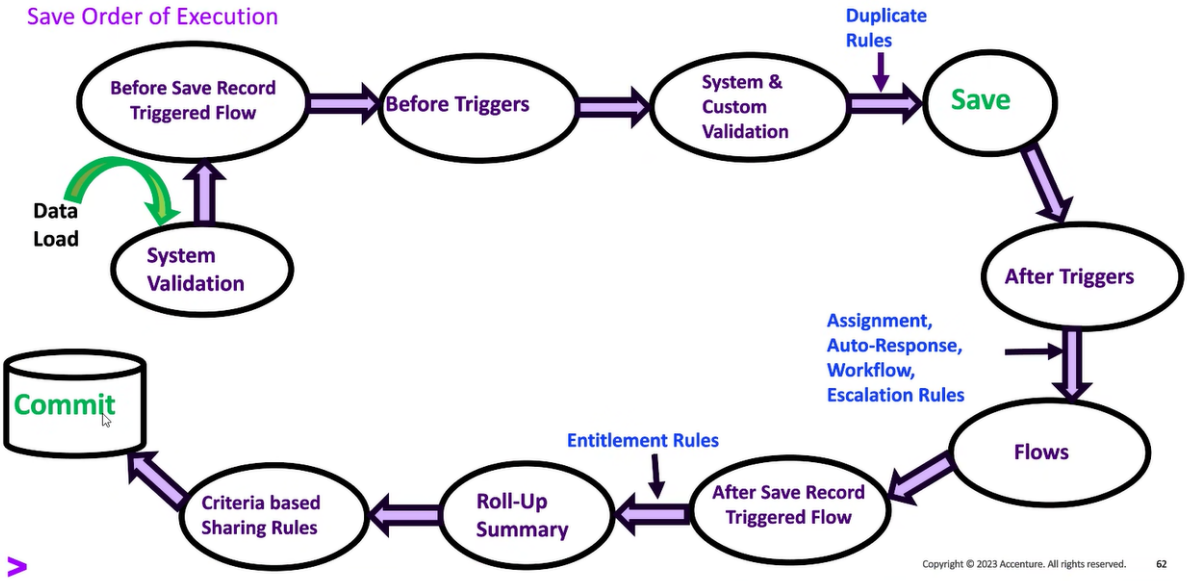
15.7 Best Practices

Key Best Practices in Apex Trigger

1. One Trigger Per Object & One Handler per Trigger
2. Trigger Handler Pattern
3. Bulkify Your Code
4. Context-Specific Handler Methods
5. Avoid Hardcoding IDs
6. Use Collections for Efficient Operations
7. Handle Recursion
8. Comprehensive Testing
9. Error Handling
10. Use Apex Naming Conventions
11. Use Custom Settings/Metadata

15.8 Order of Execution

Save Order of Execution



Order of Execution - Steps

Original Record Loaded from Database

- The original record is loaded from the database (or initialized for an insert operation).

System Validation Rules

- System validation rules are applied (e.g., required fields, field formats).

Before Save Record-Triggered Flows

- Triggered before the records are saved to the database.

Before Triggers

- Before triggers are executed. These are triggers that execute before any records are saved to the database.
  - before insert
  - before update
  - before delete

Custom Validation Rules

- Runs most system validation steps again, Custom validation rules are evaluated. Any validation errors will prevent the record from being saved.

Order of Execution - Steps

Duplicate Rules

- Duplicate rules are applied to prevent duplicate records according to defined matching rules.

Record Saved (But Not Committed)

- The record is saved to the database but not yet committed.

Order of Execution - Steps

Entitlement Rules

- Entitlement rules are applied to cases to determine if they meet the criteria for entitlements.

Roll-Up Summary Fields

- Parent record roll-up summary fields are updated (if applicable).

Criteria-Based Sharing Rules

- Criteria-based sharing rules are applied.

Commit

- The transaction is committed to the database.

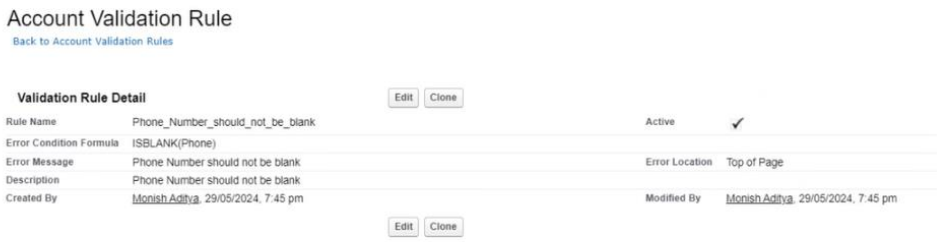
Post-Commit Logic

- Apex after triggers can include asynchronous operations, such as sending emails and enqueuing jobs for processing.

Demo

15.8 Order of Execution

Creating an Account Trigger with ‘before insert event’ to update Name to null. Considering Validation Rule is already there on Account, else create as per below.

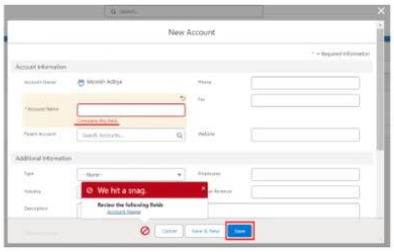


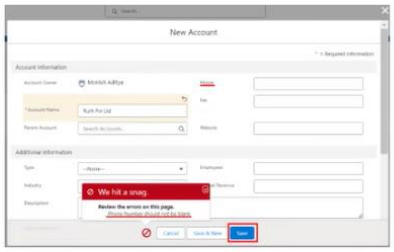
Solution:

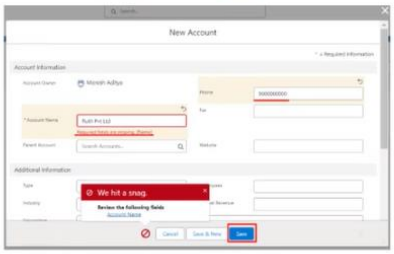
```
trigger AccountTrigger on Account (before insert) {
   for(Account accRecord : Trigger.new){
      accRecord.Name = null;
   }
}
```

Demo

15.8 Order of Execution







15.9 Static variables to prevent Trigger Recursion

Definition

- Trigger recursion can lead to infinite loops and exceeding governor limits.
- To prevent this, can use static variables in an Apex class.
- Static variables retain their value throughout the context of a single transaction, which makes them useful for managing state and ensuring certain pieces of code are only executed once.

Demo

15.9 Static variables to prevent Trigger Recursion

Create a Trigger to update the Account owner same as Opportunity owner whenever the Opportunity owner is updated & vice-versa.

Solution for Trigger Recursion:

Account:

```
trigger AccountTrigger on Account (after update) {
        AccountTriggerHandler.setOpportunityOwner(Trigger.newMap);
}
```

```
public class AccountTriggerHandler {
    public static void setOpportunityOwner(Map<Id, Account> accNewMap) {
        List<Opportunity> oppList = [Select Id, AccountId, OwnerId FROM Opportunity WHERE AccountId IN:accNewMap.keySet()];
        for(Opportunity oppRec : oppList) {
            oppRec.OwnerId = accNewMap.get(oppRec.AccountId).OwnerId;
        }
        update oppList;
    }
}
```

Demo

15.9 Static variables to prevent Trigger Recursion

Opportunity:

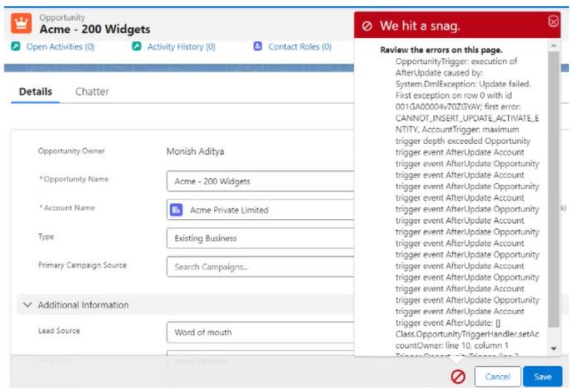
```
trigger OpportunityTrigger on Opportunity (after update) {
    OpportunityTriggerHandler.setAccountOwner(Trigger.new);
}
```

```
public class OpportunityTriggerHandler {
    public static boolean isExecuted = false;
    public static void setAccountOwner(List<Opportunity> oppNewList) {
            Map<Id,Account> accNewMap = new Map<Id,Account>();
            for(Opportunity oppRec : oppNewList) {
                accNewMap.put(oppRec.AccountId,new Account(Id=oppRec.AccountId, OwnerId=oppRec.OwnerId));
            }
            update accNewMap.values();
    }
}
```

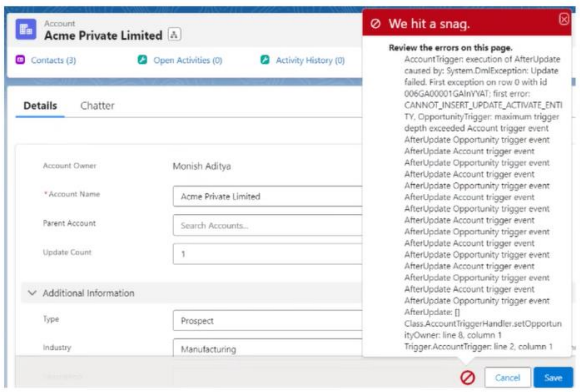
Demo

15.9 Static variables to prevent Trigger Recursion

On update of Opportunity Record :



On update of Account Record:



Demo

15.9 Static variables to prevent Trigger Recursion

Opportunity:

```
trigger OpportunityTrigger on Opportunity (after update) {
    OpportunityTriggerHandler.setAccountOwner(Trigger.new);
}
```

```
public class OpportunityTriggerHandler {
    public static boolean isExecuted = false;
    public static void setAccountOwner(List<Opportunity> oppNewList) {
        if(!isExecuted)
        {
            isExecuted = true;
            Map<Id,Account> accNewMap = new Map<Id,Account>();
            for(Opportunity oppRec : oppNewList) {
                accNewMap.put(oppRec.AccountId,new Account(Id=oppRec.AccountId, OwnerId=oppRec.OwnerId));
            }
            Database.update(accNewMap.values(),false);
        }
    }
}
```

If the Account Owner is updated, then its related opportunity owner should also get updated. And If the Opportunity owner is updated then its account owner should also gets updated automatically with the help of trigger.

To implement above scenario, We have 2 triggers one on the Account Object and other on the Opportunity object.

```
trigger AccountTrigger on Account(after update)
{
after
}

trigger oppTrigger on Opportunity(after update)
{

}
```